How the PetHaven system works

PetHaven inventory prototype — a plain-language guide to what happens when customers buy in-store or online

The big picture

PetHaven has **five shops** in Sydney and an **online store**. Three separate computer systems run the business:

1. **The store system** — the tills and stock records in the five shops. It always knows exactly what's on each shelf.
2. **The supplier delivery system** — records when suppliers drop off new stock at a shop.
3. **The online store** — the website. It shows customers **one number per product**: how many are available across all five shops combined.

The business problem is that this number isn't live. Before this project it was updated only **once a day**, so it could show stock that no shop had. The prototype now corrects it with a **refresh** that runs automatically **every 3 minutes**, and checks real stock before payment. Between refreshes it can still be out of date ("stale"), and the warehouse measures that.

Behind all three systems sits a **data warehouse**: one central record of every stock movement in the business. It's the business's record book: it watches and writes everything down, and all the reports read from it. It doesn't run the shops or the website.

All three systems know a product by the same **item number**: the dog food is *P001* in the shops, at the supplier and on the website. What differs is how each system numbers its own transactions: a shop sale has a **receipt number**, an online purchase has an **order ID**, and a delivery has a **supplier ID** and **supplier order number**. The warehouse keeps that number on every record, so anything can be traced back. The systems still name the shops differently (Parramatta is store 101 at the till, NSW-PARRA for deliveries and CP-PARRAMATTA online), so the warehouse translates those into one code, *S01*.

1. A customer buys in-store

> *Example: a customer at Parramatta buys 2 bags of dog food at the till.*

1. **The shop's stock drops straight away.** Parramatta had 37 bags; now it has 35.
2. **The sale is copied to the warehouse automatically,** in the same instant:
   - **Copy:** the sale is copied exactly as the till recorded it, with its receipt number.
   - **Translate:** the store number is converted into the warehouse's code (Parramatta becomes S01). The dog food is already P001 everywhere.
   - **Check:** if the warehouse doesn't know the item yet (for example a brand-new product nobody has added to its product list), the record is set aside with the note "Unknown item", instead of being guessed. It loads as soon as someone adds the item.
   - **Save:** the warehouse records "Parramatta, dog food, −2, at this time, receipt 29".
3. **The website doesn't change.** It still says 183 available, even though there are really 181. This is the out-of-date number the project is about, and it stays that way until the next refresh.

If the customer buys several things at once, they all go on one receipt. If even one item isn't on the shelf, the whole sale is refused.

2. A customer buys online

Example: a customer in Bondi buys 2 bags of dog food online.

1. **They add items to their bag.** The website only allows what its number says is in stock. Nothing is set aside yet, and the website number doesn't change. Bags never expire.
2. **They choose where to collect.** The website lists every shop that has their items, best first. Here Bondi Junction has the dog food and is closest, so it's top of the list.
3. **They press checkout. Before any money is taken,** the system checks the **real** stock at the chosen shop. Bondi has enough.
4. **Payment goes through and the order is created.** Bondi sets the 2 bags aside behind the counter: 2 fewer on the shelf, 2 more "reserved".
5. **The website number drops straight away** (183 → 181). The website knows about its own sales.
6. **The warehouse records the reservation** automatically, as with shop sales.
7. **The customer collects** at Bondi, and the reserved bags leave the shop.

If the customer cancels before collecting, the items go back on the shelf. If they never come, the overdue clean-up, which staff run by hand, cancels any order older than 3 days and puts the items back on the shelf. Either way, the website only finds out at the next refresh.

Two customers, one item left: putting something in a bag doesn't hold it. Whoever checks out first gets it. The other customer's checkout is stopped before payment, and the item stays in their bag.

3. An online customer buys items that are in different shops

Example: a Bondi customer buys a squeaky duck and an aquarium kit.

Before checkout, the customer sees the shops they can collect from. Only shops that have at least one of the items are offered, best first:

- **Newtown** has both items, so nothing needs moving. This is the top option.
- **Bondi** has the duck but not the kit, so the kit would be sent over from another shop.
- A shop with neither item isn't offered at all.

Say the customer chooses **Bondi** because it's closer. Payment goes through, Bondi sets the duck aside, and **Newtown** sets the kit aside. The Newtown kit then has to be **transferred**:

1. **Sent:** Newtown sends the kit. While it's on the way, it doesn't count as stock in any shop.
2. **Received:** Bondi receives it and sets it aside with the duck.
3. **Collected:** only now can the customer collect, because the order is ready only when **every** item has arrived. The system won't let them collect half an order, and the order can't be cancelled while something is on the way.

Every step (set aside, sent, received, collected) is recorded in the warehouse, so the reports always show where each item is: *waiting to be sent*, *in transit* or *ready for collection*.

4. The website shows stock that isn't really there

This is what is left of the business problem between refreshes.

> *Example from the sample data: the website was refreshed when there were 7 aquarium kits. Since then, 5 were sold in the shops. The website still says 7, but only 2 are really left — one at Newtown and one at Penrith.*

1. A Bondi customer puts **3 kits** in their bag. The website says 7, so it lets them.
2. They press checkout. **Before payment**, the system checks the real shop stock and finds that no shop has 3 kits. There are only 2 in total.
3. **Checkout is blocked.** The customer isn't charged, nothing is set aside, and they're shown which item can't be supplied. Their bag stays open, so they can remove it and buy the rest.
4. The warehouse records this as a **blocked checkout**, and **Report 3** lists it: *"website showed 7, really 2: stock sold since the last refresh, website number was out of date."*

A checkout can also be blocked when there's enough in total but **no single shop** has the full amount — for example, 2 dog beds when the shops have 1 each. Report 3 gives that reason separately: *"enough stock in total but no single store had enough."*

Why this matters: nobody is ever charged for something that can't be supplied. But the customer was still shown stock that didn't exist and walked away disappointed: a potential lost sale. That's the cost of the out-of-date number, and the reports measure it.

5. The website refresh puts the website right

The website refresh runs automatically every 3 minutes. (It can also be run by hand at any time, which is how the demonstration shows it on cue.) Each time, it:

1. Asks the shops' system how many of each product are really on the shelves, across all five shops.
2. Updates the website with those numbers.
3. The warehouse then writes down what the website refresh did: a **before and after** for every number that changed (for example, *"aquarium kit: website 7 → 2"*), what was sold or delivered by suppliers since the last refresh, and whether its records agree with every shop's own records.

The website gets its number straight from the shops, not from the warehouse. The warehouse is the record book for reports; it doesn't run the business.

Right after a website refresh the website is correct again, until the next in-store sale or supplier delivery makes it stale — for at most 3 minutes.

In one line each

Situation	What happens
In-store sale	Shelf drops instantly, warehouse updated instantly, website unchanged until refresh
Supplier delivery	Shelf rises instantly, warehouse updated instantly, website unchanged until refresh
Online order	Customer picks from the shops that have their items; real stock checked before payment; set aside; website drops instantly
Items in different shops	Each item set aside where it's found, sent to the chosen shop, collected together once all have arrived
Two customers, one item left	Whoever checks out first gets it; the other is stopped before payment and keeps it in their bag
Not collected in 3 days	Cancelled when staff run the overdue clean-up, items back on the shelf
Website shows stock that's gone	Checkout blocked before payment, no charge, recorded as a potential lost sale in Report 3
Website refresh (every 3 minutes, or by hand)	Website corrected straight from the shops' shelf totals; the warehouse logs every change before → after